Interview Trick Patterns

The Questions That Separate You From the Crowd

LeetCode · GeeksForGeeks · AtCoder · CodeChef

July 2026

A curated collection of "trick" problems across major platforms.
If you don't know the pattern, no amount of brute-force thinking will save you.
Know these cold.

Contents

Part I — LeetCode

25 Classic Interview Trick Problems

1 Single Number LC 136

Problem Statement

Every element in the array appears exactly twice *except one*. Find that single element. **Constraints:** $O(n)$ time, $O(1)$ space.

The Trick

XOR all elements. Every pair cancels ($x \oplus x = 0$). The singleton XOR'd with 0 is itself. **One line:** `return reduce(XOR, array)`.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int singleNumber(vector<int>& nums) {
    int res = 0;
    for (int x : nums) res ^= x;
    return res;
}
```

2 Single Number III LC 260

Problem Statement

Every element appears exactly twice except for **two** elements p and q. Find both. **Constraints:** $O(n)$ time, $O(1)$ space.

Note to Self

Everyone knows the single-XOR trick. Adding a second odd-frequency element breaks most people. The key — splitting on a set bit — feels arbitrary until you internalize *why* pairs always land in the same group.

The Trick

Step 1. XOR all $\Rightarrow$ $xor = p \oplus q$ (all pairs cancel).
Step 2. Pick any set bit of xor: `bit = xor & (-xor)`.
Step 3. Partition on that bit: p and q land in different groups; every pair lands in the *same* group (both copies share any given bit).
Step 4. XOR each group independently $\Rightarrow$ get p and q.

Complexity

Time: $O(n)$, two passes. **Space:** $O(1)$

```cpp
vector<int> singleNumber(vector<int>& nums) {
    int xorAll = 0;
    for (int x : nums) xorAll ^= x;
    int bit = xorAll & (-xorAll);   // lowest set bit where p != q
    int p = 0, q = 0;
    for (int x : nums) {
        if (x & bit) p ^= x;
        else         q ^= x;
    }
    return {p, q};
}
```

3 Missing Number LC 268

Problem Statement

Given an array of n distinct numbers in range $[0, n]$, return the missing one. **Constraints:** Could you implement it in $O(1)$ extra space?

The Trick

XOR indices $0 \ldots n$ with all array values. Everything cancels except the missing index. Alternatively: expected sum $= n(n+1)/2$, subtract actual sum.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int missingNumber(vector<int>& nums) {
    int n = nums.size(), res = n;
    for (int i = 0; i < n; i++) res ^= i ^ nums[i];
    return res;
}
```

4 Set Mismatch LC 645

Problem Statement

An array of n integers in $[1, n]$ has one number duplicated (D) and one missing (M). Find $[D, M]$. **Constraints:** See if you can do it without a hash set.

The Trick

Two equations, two unknowns:
- Sum difference: $\sum a_i - n(n+1)/2 = D - M$
- XOR difference: $\bigoplus a_i \oplus \bigoplus_{i=1}^{n} i = D \oplus M$

From $D - M$ and $D \oplus M$: isolate D and M via bit-split (same as LC 260).

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
vector<int> findErrorNums(vector<int>& nums) {
    int n = nums.size(), xorAll = 0;
    long long diff = 0;
    for (int i = 0; i < n; i++) {
        xorAll ^= nums[i] ^ (i + 1);
        diff   += nums[i] - (i + 1);   // = D - M
    }
    int bit = xorAll & (-xorAll);
    int x = 0, y = 0;
    for (int i = 0; i < n; i++) {
        if (nums[i] & bit) x ^= nums[i]; else y ^= nums[i];
        if ((i+1)   & bit) x ^= (i+1);   else y ^= (i+1);
    }
    // x and y are {D,M}; use diff to tell which
    return ((long long)x - y == diff) ? vector<int>{x, y} : vector<int>{y, x};
}
```

5 Find the Duplicate Number LC 287

Problem Statement

Array of $n+1$ integers, each in $[1, n]$, exactly one duplicate. **Constraints:** Must not modify the array. $O(1)$ extra space. $O(n)$ time.

Note to Self

The obvious approaches all fail: sorting modifies the array; a hash set uses $O(n)$ space; binary search on value-range gives $O(n \log n)$. The breakthrough is realizing the array is a *linked list in disguise*.

The Trick

Treat index i as a node, `nums`$[i]$ as the `next` pointer. Values in $[1, n]$ guarantee we never leave the array. With $n+1$ values mapping into n slots, the Pigeonhole Principle guarantees a cycle, and the duplicate is the **cycle entry point**. Run Floyd's cycle detection.

Floyd's Two Phases

Phase 1 (detect): `slow` moves 1 step, `fast` moves 2. They meet inside the cycle.
Phase 2 (locate): Reset `slow` to start; move both 1 step. Meeting point = cycle entry = duplicate.

Proof of Phase 2: Let μ = distance to cycle start, λ = cycle length. At the Phase 1 meeting, `slow` has taken d steps where $d \equiv 0 \pmod{\lambda}$. After resetting `slow` to index 0 and advancing both by μ steps: `slow` reaches the cycle start; `fast` (from d in the cycle) is at $d + \mu \equiv \mu \pmod{\lambda}$, which is also the cycle start. ✓

Complexity

Time: $O(n)$ **Space:** $O(1)$ Array unmodified.

```cpp
int findDuplicate(vector<int>& nums) {
    int slow = nums[0], fast = nums[0];
    do { slow = nums[slow]; fast = nums[nums[fast]]; } while (slow != fast);
    slow = nums[0];
    while (slow != fast) { slow = nums[slow]; fast = nums[fast]; }
    return slow;
}
```

6 Linked List Cycle II LC 142

Problem Statement

Given a linked list that may contain a cycle, return the **node where the cycle begins**. Return `null` if no cycle. **Constraints:** $O(1)$ space.

The Trick

Phase 1: slow (+1) and fast (+2) until they meet (proves cycle exists).
Phase 2: Reset slow to head; advance both by 1 until they meet. That meeting point is the cycle start. The mathematical proof is identical to LC 287 above.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```
ListNode* detectCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) { slow = slow->next; fast = fast->next; }
            return slow;
        }
    }
    return nullptr;
}
```

7 Intersection of Two Linked Lists LC 160

Problem Statement

Return the node at which two singly-linked lists intersect (by reference). Return `null` if no intersection. **Constraints:** $O(1)$ space, no modification.

The Trick

Run two pointers, one per list. When a pointer reaches `null`, redirect it to the *other* list's head. They meet at the intersection after traversing $a + b - c$ steps each (where c = shared tail length).

Complexity

Time: $O(m + n)$ **Space:** $O(1)$

```
ListNode* getIntersectionNode(ListNode* headA, ListNode* headB) {
    ListNode *a = headA, *b = headB;
    while (a != b) {
        a = (a == nullptr) ? headB : a->next;
        b = (b == nullptr) ? headA : b->next;
    }
    return a;
}
```

8 Sort Colors LC 75 (Dutch National Flag)

Problem Statement

Sort an array of 0s, 1s, and 2s in-place in a **single pass**. **Constraints:** $O(1)$ extra space.

The Trick

Three pointers: `lo`, `mid`, `hi`. Invariant: $[0, lo) = 0$s, $[lo, mid) = 1$s, $(hi, n-1] = 2$s, $[mid, hi] =$ unseen.

- `a[mid]==0`: swap(`lo`, `mid`), advance both.
- `a[mid]==1`: advance `mid`.
- `a[mid]==2`: swap(`mid`, `hi`), retreat `hi`. **Do NOT** advance `mid` — the swapped element is unseen.

Complexity

Time: $O(n)$ single pass **Space:** $O(1)$

```cpp
void sortColors(vector<int>& a) {
    int lo = 0, mid = 0, hi = (int)a.size() - 1;
    while (mid <= hi) {
        if      (a[mid] == 0) swap(a[lo++], a[mid++]);
        else if (a[mid] == 1) mid++;
        else                  swap(a[mid], a[hi--]);
    }
}
```

9 Move Zeroes LC 283

Problem Statement

Move all zeroes to the end while maintaining the relative order of non-zero elements. In-place, minimize total operations.

The Trick

Slow pointer `j` tracks the next write position. Fast pointer `i` scans. When `a[i] != 0`, write it to `a[j++]`. After the scan, fill `a[j..n-1]` with zeros.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
void moveZeroes(vector<int>& a) {
    int j = 0;
    for (int i = 0; i < (int)a.size(); i++)
        if (a[i] != 0) a[j++] = a[i];
    while (j < (int)a.size()) a[j++] = 0;
}
```

10 Remove Duplicates from Sorted Array LC 26

Problem Statement

Remove duplicates in-place from a sorted array, returning the count of unique elements. **Constraints:** $O(1)$ extra space.

The Trick

Slow pointer `j` = position of last unique written. Fast pointer `i` advances. When `a[i] != a[j]`, write it to `a[++j]`.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int removeDuplicates(vector<int>& a) {
    if (a.empty()) return 0;
    int j = 0;
    for (int i = 1; i < (int)a.size(); i++)
        if (a[i] != a[j]) a[++j] = a[i];
    return j + 1;
}
```

11 Majority Element LC 169

Problem Statement

Find the element appearing more than $\lfloor n/2 \rfloor$ times. Guaranteed to exist. **Constraints:** $O(1)$ space.

The Trick

Boyer-Moore Voting: Maintain a `candidate` and a `count`. Each non-matching element decrements `count`; at 0, adopt the current element. The majority has more votes than all others combined, so it always survives.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int majorityElement(vector<int>& nums) {
    int cand = nums[0], cnt = 1;
    for (int i = 1; i < (int)nums.size(); i++) {
        if (cnt == 0)           { cand = nums[i]; cnt = 1; }
        else if (nums[i]==cand)   cnt++;
        else                      cnt--;
    }
    return cand;
}
```

12 Majority Element II LC 229

Problem Statement

Find all elements appearing more than $\lfloor n/3 \rfloor$ times. **Constraints:** $O(1)$ space. Note: at most **two** such elements can exist.

The Trick

Extend Boyer-Moore to **two candidates**. For each element: match cand1 or cand2 (increment), claim an empty slot, or decrement *both* counts (one cancellation). After one pass, verify both candidates with a count pass. **Why at most two?** Three elements each $> n/3$ would exceed n total — contradiction.

Complexity

Time: $O(n)$, two passes. **Space:** $O(1)$

```cpp
vector<int> majorityElement(vector<int>& nums) {
    int c1 = 0, c2 = 1, n1 = 0, n2 = 0;   // c1 != c2 initially
    for (int x : nums) {
        if      (x == c1)  n1++;
        else if (x == c2)  n2++;
        else if (n1 == 0)  { c1 = x; n1 = 1; }
        else if (n2 == 0)  { c2 = x; n2 = 1; }
        else               { n1--; n2--; }
    }
    n1 = n2 = 0;
    for (int x : nums) { if (x==c1) n1++; else if (x==c2) n2++; }
    vector<int> res;
    int n = nums.size();
    if (n1 > n/3) res.push_back(c1);
    if (n2 > n/3) res.push_back(c2);
    return res;
}
```

13 Next Permutation LC 31

Problem Statement

Rearrange the array into the next lexicographically greater permutation in-place. If already the largest, rearrange to the smallest (ascending). $O(1)$ extra space.

Note to Self

I spent 45 minutes on this in a mock interview and gave up. The three-step algorithm feels arbitrary until you understand the "rightmost descent" framing.

The Trick

Step 1. Find the rightmost index i with $a[i] < a[i+1]$ (rightmost descent). If none, the array is the last permutation; reverse it.
Step 2. Find the rightmost $j > i$ with $a[j] > a[i]$. Swap $a[i]$ and $a[j]$.
Step 3. Reverse the suffix $a[i+1 \ldots]$.

Why it works

Everything right of i is already descending (maximum arrangement of those elements). We *must* increase $a[i]$ by the smallest possible amount (Step 2 picks the smallest value greater than $a[i]$ from the descending suffix). Then we minimise the tail by reversing it to ascending order (Step 3).

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
void nextPermutation(vector<int>& a) {
    int n = a.size(), i = n - 2;
    while (i >= 0 && a[i] >= a[i+1]) i--;
    if (i >= 0) {
        int j = n - 1;
        while (a[j] <= a[i]) j--;
        swap(a[i], a[j]);
    }
    reverse(a.begin() + i + 1, a.end());
}
```

14 Rotate Array LC 189

Problem Statement

Rotate array of n integers to the right by k steps. **Constraints:** $O(1)$ extra space.

The Trick

Triple reverse:

1. Reverse the entire array.
2. Reverse the first k elements.
3. Reverse the remaining $n - k$ elements.

After full reverse, the last k elements (which should go to the front) are in the first k positions but backwards. Two targeted reverses fix both halves.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
void rotate(vector<int>& nums, int k) {
    int n = nums.size(); k %= n;
    reverse(nums.begin(), nums.end());
    reverse(nums.begin(), nums.begin() + k);
    reverse(nums.begin() + k, nums.end());
}
```

15 Product of Array Except Self LC 238

Problem Statement

Return an array `output` where `output[i]` is the product of all elements except `nums[i]`. **Constraints:** $O(n)$ time, $O(1)$ extra space (output array doesn't count). **Division not allowed.**

Note to Self

The no-division constraint kills the obvious approach. Most people reach for division anyway and get stuck. The two-pass prefix/suffix trick is the canonical solution.

The Trick

Left pass: fill `out[i]` with the product of all elements to the *left* of i.
Right pass: traverse right-to-left, multiply `out[i]` by a running right-product (product of all elements to the *right* of i).
The output array serves as the only extra space.

Complexity

Time: $O(n)$ **Space:** $O(1)$ extra (output array excluded)

```cpp
vector<int> productExceptSelf(vector<int>& a) {
    int n = a.size();
    vector<int> out(n, 1);
    // Left pass: out[i] = product of a[0..i-1]
    for (int i = 1; i < n; i++) out[i] = out[i-1] * a[i-1];
    // Right pass: multiply by product of a[i+1..n-1]
    int right = 1;
    for (int i = n-1; i >= 0; i--) { out[i] *= right; right *= a[i]; }
    return out;
}
```

16 First Missing Positive LC 41

Problem Statement

Find the smallest missing positive integer. **Constraints:** $O(n)$ time, $O(1)$ space. Example: $[3, 4, -1, 1] \Rightarrow 2$, $[7, 8, 9] \Rightarrow 1$.

The Trick

The answer is always in $[1, n+1]$, so only values in $[1, n]$ matter. **Use the array as a hash map:** value v "belongs" at index $v - 1$. Cycle-sort all values to their correct positions, then find the first mismatch.

Why the answer is in $[1, n+1]$

There are n slots. If $1, 2, \ldots, n$ are all present, the answer is $n + 1$. Otherwise some $k \in [1, n]$ is missing. Either way, we need only inspect $[1, n]$.

Complexity

Time: $O(n)$ amortised (each element moves to its slot at most once). **Space:** $O(1)$

```
int firstMissingPositive(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n; i++)
        while (a[i] >= 1 && a[i] <= n && a[a[i]-1] != a[i])
            swap(a[i], a[a[i]-1]);
    for (int i = 0; i < n; i++)
        if (a[i] != i+1) return i+1;
    return n+1;
}
```

17 Find All Numbers Disappeared in an Array LC 448

Problem Statement

Array of n integers in $[1, n]$; some elements appear twice, others are missing. Find all missing numbers without extra space ($O(1)$ extra).

The Trick

For each value $v = |a[i]|$, negate $a[v-1]$ (mark as "visited"). After the pass, any index i with $a[i] > 0$ means value $i+1$ was never seen. Use the sign bit as a boolean presence marker.

Complexity

Time: $O(n)$, two passes. **Space:** $O(1)$ extra.

```cpp
vector<int> findDisappearedNumbers(vector<int>& a) {
    for (int i = 0; i < (int)a.size(); i++) {
        int idx = abs(a[i]) - 1;
        if (a[idx] > 0) a[idx] = -a[idx];
    }
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++)
        if (a[i] > 0) res.push_back(i + 1);
    return res;
}
```

18 Trapping Rain Water LC 42

Problem Statement

Given an elevation map, compute the total trapped water after rain. **Constraints:** $O(1)$ extra space.

The Trick

Two pointers `l`, `r` with running `maxL` and `maxR`. If $h[l] \le h[r]$: water at l is `maxL`$-h[l]$ (the right side is guaranteed $\ge h[r] \ge h[l]$ so it won't be the limiting factor). Process l and advance. Otherwise process r symmetrically and retreat.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int trap(vector<int>& h) {
    int l = 0, r = (int)h.size()-1, mxL = 0, mxR = 0, water = 0;
    while (l < r) {
        if (h[l] <= h[r]) { mxL = max(mxL, h[l]); water += mxL - h[l++]; }
        else              { mxR = max(mxR, h[r]); water += mxR - h[r--]; }
    }
    return water;
}
```

19 Maximum Product Subarray LC 152

Problem Statement

Find the contiguous subarray with the largest product. Array may contain negatives and zeros.

Note to Self

I always defaulted to Kadane's and got WA. Products are not like sums: a large negative $\times$ a large negative $=$ a large positive. You *must* track both the running maximum and the running minimum.

The Trick

At every index maintain `curMax` and `curMin` ending here. If the current element is negative, **swap** `curMax` and `curMin` before updating (a negative flips the ordering).

$$\texttt{curMax} = \max(a[i],\ a[i] \cdot \texttt{curMax}) \quad \texttt{curMin} = \min(a[i],\ a[i] \cdot \texttt{curMin})$$

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int maxProduct(vector<int>& a) {
    int curMax = a[0], curMin = a[0], ans = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        if (a[i] < 0) swap(curMax, curMin);
        curMax = max(a[i], curMax * a[i]);
        curMin = min(a[i], curMin * a[i]);
        ans = max(ans, curMax);
    }
    return ans;
}
```

20 Container With Most Water LC 11

Problem Statement

Given heights $h[0 \ldots n-1]$, find two indices $l < r$ that maximise $(r-l) \times \min(h[l], h[r])$.

The Trick

Start with the widest possible container ($l = 0$, $r = n-1$). Always move the **shorter** pointer inward: moving the taller pointer can only decrease width without any possibility of increasing the height limit (which is capped by the shorter bar anyway). Moving the shorter pointer might find a taller bar, potentially increasing the area.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int maxArea(vector<int>& h) {
    int l = 0, r = (int)h.size()-1, ans = 0;
    while (l < r) {
        ans = max(ans, min(h[l], h[r]) * (r - l));
        if (h[l] < h[r]) l++; else r--;
    }
    return ans;
}
```

21 3Sum LC 15

Problem Statement

Find all unique triplets in the array that sum to zero. No duplicate triplets.

Note to Self

The trap here isn't the algorithm — it's the **deduplication**. After sorting, you must skip duplicate values at all three pointer positions, otherwise you'll emit the same triplet multiple times. Easy to get wrong under contest pressure.

The Trick

Sort the array. Fix element $a[i]$ and run two pointers $l = i + 1$, $r = n - 1$:

- $a[i] + a[l] + a[r] == 0$: record, then skip duplicates at l and r.
- Sum too small: advance l.
- Sum too large: retreat r.

Skip duplicate values of $a[i]$ at the outer loop.

Complexity

Time: $O(n^2)$ **Space:** $O(1)$ (excluding output)

```cpp
vector<vector<int>> threeSum(vector<int>& a) {
    sort(a.begin(), a.end());
    vector<vector<int>> res;
    int n = a.size();
    for (int i = 0; i < n-2; i++) {
        if (i > 0 && a[i] == a[i-1]) continue;   // skip dup i
        int l = i+1, r = n-1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s == 0) {
                res.push_back({a[i], a[l], a[r]});
                while (l < r && a[l]==a[l+1]) l++;  // skip dup l
                while (l < r && a[r]==a[r-1]) r--;  // skip dup r
                l++; r--;
            } else if (s < 0) l++;
            else r--;
        }
    }
    return res;
}
```

22 Jump Game LC 55

Problem Statement

Given `nums[i]` = max jump length from position i, can you reach the last index?

The Trick

Track the furthest reachable index `maxReach`. At each position i: if $i >$ `maxReach`, we're stuck — return false. Otherwise update `maxReach` = $\max($`maxReach`$, i +$ `nums`$[i])$.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
bool canJump(vector<int>& nums) {
    int maxReach = 0;
    for (int i = 0; i < (int)nums.size(); i++) {
        if (i > maxReach) return false;
        maxReach = max(maxReach, i + nums[i]);
    }
    return true;
}
```

23 Gas Station LC 134

Problem Statement

n gas stations in a circle. `gas[i]` = gas available, `cost[i]` = gas to travel to next station. Find the starting station index to complete the circuit, or -1.

The Trick

Key insight 1: If total gas $\geq$ total cost, a solution is guaranteed.
Key insight 2: Whenever the running tank drops below 0, reset it to 0 and mark the next station as the new candidate starting point.
The station after the last "reset" is the answer.

Why the last reset gives the correct start

If the running sum goes negative at position k, no station in $[start, k]$ can serve as a valid starting point (they all inherited an insufficient prefix). Starting fresh at $k+1$ is the greedy choice. The global sum guarantee means this candidate will succeed on the full circuit.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < (int)gas.size(); i++) {
        int diff = gas[i] - cost[i];
        total += diff; tank += diff;
        if (tank < 0) { start = i + 1; tank = 0; }
    }
    return (total >= 0) ? start : -1;
}
```

24 Candy LC 135

Problem Statement

n children in a row with ratings. Each child gets ≥ 1 candy. Children with higher ratings than their *neighbour* must get more candies. Minimise total candies.

Note to Self

A single left-to-right pass is not enough. You need to consider both neighbours. The two-pass approach elegantly separates the left and right constraints.

The Trick

Left-to-right pass: If `rating[i] > rating[i-1]`, then `candy[i] = candy[i-1]+1`; else `candy[i] = 1`.
Right-to-left pass: If `rating[i] > rating[i+1]`, then `candy[i] = max(candy[i], candy[i+1]+1)`.
Take the `max` to satisfy *both* the left and right constraints.

Complexity

Time: $O(n)$ **Space:** $O(n)$

```cpp
int candy(vector<int>& r) {
    int n = r.size();
    vector<int> c(n, 1);
    for (int i = 1; i < n; i++)
        if (r[i] > r[i-1]) c[i] = c[i-1] + 1;
    for (int i = n-2; i >= 0; i--)
        if (r[i] > r[i+1]) c[i] = max(c[i], c[i+1] + 1);
    return accumulate(c.begin(), c.end(), 0);
}
```

25 Merge Intervals LC 56

Problem Statement

Given a list of intervals, merge all overlapping intervals and return the result.

The Trick

Sort by start time. Scan left to right: if the current interval overlaps with the last merged interval (`cur.start` $\leq$ `last.end`), extend `last.end`. Otherwise push `cur` as a new interval.

Complexity

Time: $O(n \log n)$ (dominated by sort) **Space:** $O(1)$ extra

```cpp
vector<vector<int>> merge(vector<vector<int>>& iv) {
    sort(iv.begin(), iv.end());
    vector<vector<int>> res;
    for (auto& cur : iv) {
        if (!res.empty() && cur[0] <= res.back()[1])
            res.back()[1] = max(res.back()[1], cur[1]);
        else
            res.push_back(cur);
    }
    return res;
}
```

Part II — GeeksForGeeks

25 Classic Indian Interview Favourites

26 Find Two Numbers with Odd Frequency GFG

Problem Statement

Every element appears an even number of times except exactly two (p and q). Find both. **Constraints:** $O(1)$ space.

The Trick

XOR all elements $\Rightarrow p \oplus q$. Pick the lowest set bit (they differ there). Partition array on that bit: pairs land in the same group and cancel; p and q land in opposite groups. XOR each group independently.

Complexity

Time: $O(n)$, two passes. **Space:** $O(1)$

```
pair<int,int> twoOddFreq(vector<int>& a) {
    int xr = 0; for (int x : a) xr ^= x;
    int bit = xr & (-xr);
    int p = 0, q = 0;
    for (int x : a) { if (x & bit) p ^= x; else q ^= x; }
    return {p, q};
}
```

27 Find Missing and Duplicate GFG

Problem Statement

Array of n integers in $[1, n]$: one value appears twice (D), one is absent (M). Find both. **Constraints:** $O(1)$ space, no modification.

The Trick

$\sum a - n(n+1)/2 = D - M$ and $\bigoplus a \oplus \bigoplus_{i=1}^{n} i = D \oplus M$.
Use the sum equation to distinguish D from M once the XOR bit-split gives both candidates.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```
pair<int,int> findDupMissing(vector<int>& a) {
    int n = a.size(), xr = 0; long long diff = 0;
    for (int i = 0; i < n; i++) { xr ^= a[i]^(i+1); diff += a[i]-(i+1); }
    int bit = xr & (-xr), x = 0, y = 0;
    for (int i = 0; i < n; i++) {
        if (a[i]   & bit) x ^= a[i]; else y ^= a[i];
        if ((i+1)  & bit) x ^= i+1;  else y ^= i+1;
    }
    return ((long long)x - y == diff) ? make_pair(x,y) : make_pair(y,x);
}
```

28 Kadane's Algorithm (Maximum Subarray Sum) GFG

Problem Statement

Find the contiguous subarray with the largest sum.

The Trick

At each position, `curMax = max(a[i], curMax + a[i])`. If the running sum goes negative, discard the prefix and start fresh.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
long long maxSubarraySum(vector<int>& a) {
    long long cur = a[0], ans = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        cur = max((long long)a[i], cur + a[i]);
        ans = max(ans, cur);
    }
    return ans;
}
```

29 Stock Buy & Sell I (Single Transaction) GFG

Problem Statement

Find the maximum profit from a single buy-sell transaction. Must buy before selling.

The Trick

Track `minPrice` seen so far. At each day, profit = `price - minPrice`. Track `maxProfit` over all days.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int maxProfit(vector<int>& p) {
    int mn = p[0], ans = 0;
    for (int i = 1; i < (int)p.size(); i++) {
        ans = max(ans, p[i] - mn);
        mn  = min(mn, p[i]);
    }
    return ans;
}
```

30 Stock Buy & Sell II (Unlimited Transactions) GFG

Problem Statement

Collect the maximum profit from as many transactions as you like (sell before buying again).

The Trick

Sum every positive consecutive difference: if $p[i] > p[i-1]$, add $p[i] - p[i-1]$. This is equivalent to capturing every upward move in the price.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int maxProfit(vector<int>& p) {
    int ans = 0;
    for (int i = 1; i < (int)p.size(); i++)
        if (p[i] > p[i-1]) ans += p[i] - p[i-1];
    return ans;
}
```

31 Equilibrium Point GFG

Problem Statement

Find an index i such that the sum of all elements to the left equals the sum of all elements to the right. Return -1 if none.

The Trick

Compute `total`. Walk left to right maintaining `leftSum`. At index i: `rightSum = total - leftSum - a[i]`. If `leftSum == rightSum`, found it. Then `leftSum += a[i]`. No prefix array needed.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
int equilibriumPoint(vector<long long>& a) {
    long long total = 0; for (auto x : a) total += x;
    long long left = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        if (left == total - left - a[i]) return i + 1; // 1-indexed
        left += a[i];
    }
    return -1;
}
```

32 Leaders in an Array GFG

Problem Statement

An element is a "leader" if it is greater than all elements to its right. Find all leaders (the rightmost element is always a leader).

The Trick

Scan **right to left**, tracking `maxSoFar`. Every element greater than `maxSoFar` is a leader. Collect in reverse, then reverse output.

Complexity

Time: $O(n)$ **Space:** $O(1)$ extra

```cpp
vector<int> leaders(vector<int>& a) {
    int n = a.size(), mx = a[n-1];
    vector<int> res = {mx};
    for (int i = n-2; i >= 0; i--)
        if (a[i] >= mx) { mx = a[i]; res.push_back(mx); }
    reverse(res.begin(), res.end());
    return res;
}
```

33 Wave Array GFG

Problem Statement

Rearrange the array in a wave-like fashion such that $a[0] \geq a[1] \leq a[2] \geq a[3] \leq \ldots$ in-place.

The Trick

Sort the array. Swap every adjacent pair: $(a[0], a[1])$, $(a[2], a[3])$, ... Sorting guarantees each swapped pair satisfies the wave condition.

Complexity

Time: $O(n \log n)$ (for sort) **Space:** $O(1)$

```cpp
void waveArray(vector<int>& a) {
    sort(a.begin(), a.end());
    for (int i = 0; i+1 < (int)a.size(); i += 2)
        swap(a[i], a[i+1]);
}
```

34 Count Inversions GFG

Problem Statement

Count the number of pairs (i, j) with $i < j$ but $a[i] > a[j]$. **Constraints:** Better than $O(n^2)$.

Note to Self

The "merge sort trick" is the standard technique here. The insight is that during the merge step, whenever you pick from the right half, every remaining element in the left half forms an inversion with it.

The Trick

Modified merge sort: during the merge, when an element from the right subarray is smaller than an element from the left, *all remaining elements in the left sub-array* form inversions with it. Add `(mid - l + 1)` to the count.

Complexity

Time: $O(n \log n)$ **Space:** $O(n)$ (merge buffer)

```cpp
long long mergeCount(vector<int>& a, int l, int mid, int r) {
    vector<int> tmp;
    long long inv = 0;
    int i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) tmp.push_back(a[i++]);
        else { inv += mid - i + 1; tmp.push_back(a[j++]); }
    }
    while (i <= mid) tmp.push_back(a[i++]);
    while (j <= r)   tmp.push_back(a[j++]);
    for (int k = l; k <= r; k++) a[k] = tmp[k - l];
    return inv;
}
long long countInv(vector<int>& a, int l, int r) {
    if (l >= r) return 0;
    int mid = (l + r) / 2;
    return countInv(a,l,mid) + countInv(a,mid+1,r) + mergeCount(a,l,mid,r);
}
```

35 Largest Subarray with 0 Sum GFG

Problem Statement

Find the length of the largest subarray with sum equal to 0.

The Trick

If prefix sums $P[i] = P[j]$ for $i < j$, then the subarray $[i+1, j]$ has sum 0. Store the *first* occurrence of each prefix sum in a hash map. If the same prefix sum appears again at index j, the subarray length is $j - \texttt{firstSeen}$.

Complexity

Time: $O(n)$ average **Space:** $O(n)$

```cpp
int maxLen(vector<int>& a) {
    unordered_map<int,int> seen; seen[0] = -1;
    int sum = 0, ans = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        sum += a[i];
        if (seen.count(sum)) ans = max(ans, i - seen[sum]);
        else                 seen[sum] = i;
    }
    return ans;
}
```

36 Subarray with Given Sum GFG

Problem Statement

Given a non-negative array, find a contiguous subarray with sum equal to S. Return the 1-indexed start and end positions, or $[-1]$.

The Trick

Sliding window: expand right to add elements, shrink left when sum exceeds S. Works because all values are non-negative (sum is monotone in window size).

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
vector<int> subarraySum(vector<int>& a, long long s) {
    long long cur = 0; int l = 0;
    for (int r = 0; r < (int)a.size(); r++) {
        cur += a[r];
        while (cur > s && l <= r) cur -= a[l++];
        if (cur == s) return {l+1, r+1};
    }
    return {-1};
}
```

37 Minimum Number of Platforms GFG

Problem Statement

Given arrival and departure times of trains, find the minimum number of platforms needed so no train waits. Classic scheduling problem.

The Trick

Sort `arr[]` and `dep[]` independently. Use two pointers: advance the arrival pointer if the next train arrives before the earliest departure; increment platform count. Retreat the departure pointer and decrement the count otherwise. Track the running maximum.

Complexity

Time: $O(n \log n)$ (sort) **Space:** $O(1)$

```cpp
int findPlatform(vector<int> arr, vector<int> dep, int n) {
    sort(arr.begin(), arr.end());
    sort(dep.begin(), dep.end());
    int plat = 1, ans = 1, i = 1, j = 0;
    while (i < n && j < n) {
        if (arr[i] <= dep[j]) { plat++; i++; }
        else                  { plat--; j++; }
        ans = max(ans, plat);
    }
    return ans;
}
```

38 Activity Selection Problem GFG

Problem Statement

Given n activities with start and finish times, select the maximum number of non-overlapping activities.

The Trick

Sort by **finish time**. Greedily pick each activity that starts at or after the last selected activity ends. The earliest-finishing activity always leaves the most room for future activities.

Complexity

Time: $O(n \log n)$ **Space:** $O(1)$

```cpp
int activitySelection(vector<int>& start, vector<int>& end, int n) {
    vector<int> idx(n); iota(idx.begin(), idx.end(), 0);
    sort(idx.begin(), idx.end(), [&](int a, int b){ return end[a] < end[b]; });
    int cnt = 1, last = idx[0];
    for (int i = 1; i < n; i++)
        if (start[idx[i]] > end[last]) { cnt++; last = idx[i]; }
    return cnt;
}
```

39 Job Sequencing Problem GFG

Problem Statement

Each job has a deadline and a profit (takes 1 unit of time). Maximise total profit by scheduling at most one job per time slot. Jobs must finish by their deadline.

The Trick

Sort by profit descending. For each job, assign it to the **latest available slot** $\leq$ its deadline. Use a simple boolean array (or Disjoint Set Union for $O(\log n)$ per slot lookup) to track free slots.

Complexity

Time: $O(n^2)$ (naive) or $O(n \log n)$ with DSU. **Space:** $O(n)$

```cpp
int jobSequencing(vector<int>& deadline, vector<int>& profit, int n) {
    vector<int> idx(n); iota(idx.begin(), idx.end(), 0);
    sort(idx.begin(), idx.end(), [&](int a, int b){ return profit[a] > profit[b]; });
    int maxD = *max_element(deadline.begin(), deadline.end());
    vector<bool> slot(maxD + 1, false);
    int total = 0;
    for (int i : idx) {
        for (int d = deadline[i]; d >= 1; d--) {
            if (!slot[d]) { slot[d] = true; total += profit[i]; break; }
        }
    }
    return total;
}
```

40 Chocolate Distribution Problem GFG

Problem Statement

Given n chocolate packets with varying counts and m students, distribute one packet per student to minimise the difference between the maximum and minimum chocolates given.

The Trick

Sort the array. The optimal window of size m is the one where $a[i+m-1]-a[i]$ is minimised. Slide the window over the sorted array.

Complexity

Time: $O(n \log n)$ **Space:** $O(1)$

```cpp
long long findMinDiff(vector<long long>& a, int m) {
    sort(a.begin(), a.end());
    long long ans = a[m-1] - a[0];
    for (int i = 1; i + m - 1 < (int)a.size(); i++)
        ans = min(ans, a[i+m-1] - a[i]);
    return ans;
}
```

41 Pythagorean Triplet in Array GFG

Problem Statement

Determine if the array contains a Pythagorean triplet (a, b, c) with $a^2 + b^2 = c^2$.

The Trick

Square all elements, sort. For each candidate hypotenuse $c^2 = a[\text{last}]$, use two pointers on $a[0 \dots \text{last} - 1]$ to find $a^2 + b^2 = c^2$.

Complexity

Time: $O(n^2)$ **Space:** $O(1)$

```cpp
bool pythagoreanTriplet(vector<int>& a) {
    for (auto& x : a) x *= x;
    sort(a.begin(), a.end());
    int n = a.size();
    for (int k = n-1; k >= 2; k--) {
        int l = 0, r = k - 1;
        while (l < r) {
            int s = a[l] + a[r];
            if (s == a[k]) return true;
            else if (s < a[k]) l++; else r--;
        }
    }
    return false;
}
```

42 Move Negative Numbers to Front GFG

Problem Statement

Rearrange an array so all negative numbers come before all positive numbers. Relative order within each group need not be maintained. $O(1)$ space.

The Trick

Two-group Dutch National Flag: one pointer `j` tracks the boundary. Walk `i` forward; when `a[i] < 0`, swap with `a[j++]`.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
void rearrange(vector<int>& a) {
    int j = 0;
    for (int i = 0; i < (int)a.size(); i++)
        if (a[i] < 0) swap(a[i], a[j++]);
}
```

43 Sliding Window Maximum GFG

Problem Statement

Given an array and window size k, return the maximum of each window of size k.

Note to Self

The naive $O(nk)$ solution is obvious. The deque trick is the one to know. Many people reach for a max-heap ($O(n \log k)$) but the deque gives $O(n)$.

The Trick

Maintain a deque of **indices in decreasing order of their values**.

- Remove indices outside the window from the front.
- Remove indices from the back whose values are $\leq$ current (they can never be the max).
- The front of the deque is always the maximum of the current window.

Complexity

Time: $O(n)$ **Space:** $O(k)$

```cpp
vector<int> maxSlidingWindow(vector<int>& a, int k) {
    deque<int> dq;  // stores indices
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        while (!dq.empty() && a[dq.back()] <= a[i])  dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

44 Next Greater Element GFG

Problem Statement

For each element in the array, find the next greater element (to the right). Output -1 if none exists.

The Trick

Monotonic stack: Maintain a stack of elements whose "next greater" is not yet found. For each new element x: while the stack's top is $< x$, pop it and record x as its next greater element. Then push x.

Complexity

Time: $O(n)$ **Space:** $O(n)$

```cpp
vector<int> nextGreater(vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st;  // stores indices
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) {
            res[st.top()] = a[i]; st.pop();
        }
        st.push(i);
    }
    return res;
}
```

45 Kth Smallest Element in BST GFG

Problem Statement

Find the k-th smallest element in a Binary Search Tree. **Constraints:** $O(H)$ space (tree height), or $O(1)$ with Morris traversal.

The Trick

BST inorder traversal visits nodes in sorted order. Count nodes as you go; return the k-th one. Use an iterative stack to avoid $O(n)$ recursion overhead.

Complexity

Time: $O(H + k)$ **Space:** $O(H)$ with stack

```
int kthSmallest(TreeNode* root, int k) {
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }
        cur = st.top(); st.pop();
        if (--k == 0) return cur->val;
        cur = cur->right;
    }
    return -1;
}
```

46 Middle of a Linked List GFG

Problem Statement

Return the middle node of a linked list. If two middles exist, return the second.

The Trick

Slow pointer moves 1 step, fast pointer moves 2 steps. When fast reaches the end, slow is at the middle.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```
ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}
```

47 Palindrome Linked List GFG

Problem Statement

Check if a linked list is a palindrome. **Constraints:** $O(1)$ extra space.

The Trick

Find the middle (slow/fast pointers). Reverse the second half in-place. Compare first half and reversed second half node-by-node. Optionally restore the list by reversing again.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```
ListNode* reverseList(ListNode* head) {
    ListNode *prev = nullptr, *cur = head;
    while (cur) { auto nxt = cur->next; cur->next = prev; prev = cur; cur = nxt; }
    return prev;
}
bool isPalindrome(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* rev = reverseList(slow);
    ListNode *a = head, *b = rev;
    bool ok = true;
    while (b) { if (a->val != b->val) { ok = false; break; } a = a->next; b = b->next; }
    reverseList(rev);  // restore
    return ok;
}
```

48 Kth Node from End of Linked List GFG

Problem Statement

Find the k-th node from the end of a linked list in one pass.

The Trick

Two pointers: advance the first by k steps. Then advance both until the first reaches the end. The second pointer is now at the k-th node from the end.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```
ListNode* kthFromEnd(ListNode* head, int k) {
    ListNode *fast = head, *slow = head;
    for (int i = 0; i < k; i++) fast = fast->next;
    while (fast) { fast = fast->next; slow = slow->next; }
    return slow;
}
```

49 Reverse Linked List in Groups of K GFG

Problem Statement

Reverse every k nodes in a linked list. If the last group has fewer than k nodes, leave them as-is.

The Trick

Recursively: reverse the first k nodes, then attach the result of recursing on the rest. The key pointer manipulation: after reversing, the original head becomes the new tail of the reversed group; point it to the recursive result.

Complexity

Time: $O(n)$ **Space:** $O(n/k)$ recursion stack

```
ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode* cur = head; int cnt = 0;
    while (cur && cnt < k) { cur = cur->next; cnt++; }
    if (cnt < k) return head;  // fewer than k remaining: leave as-is
    // Reverse first k nodes
    ListNode *prev = nullptr, *node = head;
    for (int i = 0; i < k; i++) {
        auto nxt = node->next; node->next = prev; prev = node; node = nxt;
    }
    head->next = reverseKGroup(node, k);  // head is now the tail of this group
    return prev;  // prev is the new head
}
```

50 Detect and Remove Loop in Linked List GFG

Problem Statement

If a linked list has a cycle, remove it (without losing any nodes).

The Trick

Detect with Floyd's. **Locate** the loop start (Phase 2 reset, same as LC 142). **Remove:** traverse the loop to find the node whose `next` is the loop start (the last node in the cycle); set its `next` to `null`.

Complexity

Time: $O(n)$ **Space:** $O(1)$

```cpp
void removeLoop(ListNode* head) {
    ListNode *slow = head, *fast = head;
    bool hasCycle = false;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) { hasCycle = true; break; }
    }
    if (!hasCycle) return;
    slow = head;
    if (slow == fast) {  // loop starts at head
        while (fast->next != slow) fast = fast->next;
    } else {
        while (slow->next != fast->next) { slow = slow->next; fast = fast->next; }
    }
    fast->next = nullptr;  // break the loop
}
```

Part III — AtCoder

12 Essential Algorithmic Techniques

51 Prefix XOR for Range Queries

AtCoder / General

Problem Statement

Given an array, answer Q queries each asking for the XOR of all elements in range $[l, r]$. Preprocess in $O(n)$; each query in $O(1)$.

The Trick / Technique

Build a prefix XOR array: $\texttt{px}[0] = 0$, $\texttt{px}[i] = a[0] \oplus \cdots \oplus a[i-1]$. Range XOR $[l, r] = \texttt{px}[r+1] \oplus \texttt{px}[l]$. Follows from the same self-cancellation property as prefix sums.

Complexity

Build: $O(n)$ **Query:** $O(1)$ **Space:** $O(n)$

```cpp
vector<int> buildPrefixXOR(vector<int>& a) {
    int n = a.size();
    vector<int> px(n + 1, 0);
    for (int i = 0; i < n; i++) px[i+1] = px[i] ^ a[i];
    return px;
}
// Query [l, r] (0-indexed):
int queryXOR(vector<int>& px, int l, int r) { return px[r+1] ^ px[l]; }
```

52 Difference Array for Range Updates

AtCoder / General

Problem Statement

Given an array of n zeros and Q range-update operations (add v to all elements in $[l, r]$), reconstruct the final array. $O(1)$ per update, $O(n)$ reconstruct.

The Trick / Technique

Maintain a **difference array** d. For "add v to $[l, r]$": set $d[l]$ += v and $d[r+1]$ −= v. After all updates, the prefix sum of d gives the final array. Each update creates a "step function" that the prefix sum undoes.

Complexity

Update: $O(1)$ **Reconstruct:** $O(n)$ **Space:** $O(n)$

```cpp
struct DiffArray {
    int n; vector<long long> d;
    DiffArray(int n) : n(n), d(n+2, 0) {}
    void update(int l, int r, long long v) { d[l] += v; d[r+1] -= v; }
    vector<long long> build() {
        vector<long long> a(n);
        a[0] = d[0];
        for (int i = 1; i < n; i++) a[i] = a[i-1] + d[i];
        return a;
    }
};
```

53 Coordinate Compression

AtCoder / General

Problem Statement

You have values up to 10^9 but only $n \leq 10^5$ distinct ones. Map each value to a rank in $[0, n)$ to use them as array indices.

The Trick / Technique

Sort + deduplicate the values. For any query value, its rank is found via `lower_bound` in $O(\log n)$.

Complexity

Build: $O(n \log n)$ **Query:** $O(\log n)$

```
struct CoordCompress {
    vector<int> vals;
    void build(vector<int>& a) {
        vals = a;
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
    }
    int rank(int x) { return lower_bound(vals.begin(), vals.end(), x) - vals.begin(); }
    int size()      { return vals.size(); }
};
```

54 Sparse Table for O(1) Range Minimum Query

AtCoder / General

Problem Statement

Preprocess an array so that any range minimum (or maximum) query $[l, r]$ is answered in $O(1)$.

The Trick / Technique

Build $\texttt{sp}[k][i]$ = minimum of 2^k elements starting at i. Transition: $\texttt{sp}[k][i] = \min(\texttt{sp}[k-1][i],\ \texttt{sp}[k-1][i+2^{k-1}])$. Query $[l, r]$: let $k = \lfloor \log_2(r - l + 1) \rfloor$. The answer is $\min(\texttt{sp}[k][l],\ \texttt{sp}[k][r - 2^k + 1])$. The two overlapping windows of size 2^k together cover $[l, r]$ completely. **Overlapping is fine for idempotent operations** (min, max, gcd).

Complexity

Build: $O(n \log n)$ **Query:** $O(1)$ **Space:** $O(n \log n)$

```cpp
struct SparseTable {
    vector<vector<int>> sp; vector<int> lg;
    void build(vector<int>& a) {
        int n = a.size(), K = __lg(n) + 1;
        sp.assign(K, vector<int>(n));
        lg.resize(n + 1); lg[1] = 0;
        for (int i = 2; i <= n; i++) lg[i] = lg[i/2] + 1;
        sp[0] = a;
        for (int k = 1; k < K; k++)
            for (int i = 0; i + (1<<k) <= n; i++)
                sp[k][i] = min(sp[k-1][i], sp[k-1][i + (1<<(k-1))]);
    }
    int query(int l, int r) {
        int k = lg[r - l + 1];
        return min(sp[k][l], sp[k][r - (1<<k) + 1]);
    }
};
```

55 Contribution Technique

AtCoder / General

Problem Statement

Example: Find the sum of maximums over all subarrays of an array. Brute force is $O(n^2)$.

Note to Self

The contribution technique reframes the problem: instead of iterating over all subarrays and finding their max, ask "how many subarrays have element $a[i]$ as their maximum?" and multiply.

The Trick / Technique

For each element $a[i]$, find $L[i]$ = index of the previous greater element (or -1), and $R[i]$ = index of the next greater-or-equal element (or n). The number of subarrays where $a[i]$ is the maximum is $(i - L[i]) \times (R[i] - i)$. Use a monotonic stack to compute L and R in $O(n)$.

Complexity

Time: $O(n)$ **Space:** $O(n)$

```cpp
long long sumSubarrayMaximums(vector<int>& a) {
    int n = a.size();
    vector<int> L(n), R(n); stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) st.pop();
        L[i] = st.empty() ? -1 : st.top(); st.push(i);
    }
    while (!st.empty()) st.pop();
    for (int i = n-1; i >= 0; i--) {
        while (!st.empty() && a[st.top()] <= a[i]) st.pop();
        R[i] = st.empty() ? n : st.top(); st.push(i);
    }
    long long ans = 0;
    for (int i = 0; i < n; i++)
        ans += (long long)a[i] * (i - L[i]) * (R[i] - i);
    return ans;
}
```

56 Count Subarrays with XOR Equal to K

AtCoder / General

Problem Statement

Count the number of subarrays with XOR equal to K.

The Trick / Technique

Prefix XOR trick: subarray $[l, r]$ has XOR $= K$ iff $\texttt{px}[r+1] \oplus \texttt{px}[l] = K$, i.e., $\texttt{px}[l] = \texttt{px}[r+1] \oplus K$. Use a hash map of prefix XOR frequencies (same as the "subarray sum = K" pattern).

Complexity

Time: $O(n)$ average **Space:** $O(n)$

```cpp
int countSubarraysXorK(vector<int>& a, int K) {
    unordered_map<int,int> freq; freq[0] = 1;
    int px = 0, ans = 0;
    for (int x : a) {
        px ^= x;
        ans += freq[px ^ K];
        freq[px]++;
    }
    return ans;
}
```

57 Z-Algorithm for String Matching

AtCoder / General

Problem Statement

Given a pattern P and text T, find all occurrences of P in T in $O(|P| + |T|)$.

The Trick / Technique

Build the **Z-array** for the concatenated string $S = P + \$ + T$, where $Z[i]$ = length of the longest substring starting at $S[i]$ that is also a prefix of S. Any position i in the text portion where $Z[i] = |P|$ is a match.

Complexity

Time: $O(|P| + |T|)$ **Space:** $O(|P| + |T|)$

```cpp
vector<int> zFunction(const string& s) {
    int n = s.size(); vector<int> z(n, 0); z[0] = n;
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}
vector<int> findOccurrences(const string& pat, const string& txt) {
    string s = pat + "$" + txt;
    auto z = zFunction(s);
    vector<int> res;
    int m = pat.size();
    for (int i = m + 1; i < (int)s.size(); i++)
        if (z[i] == m) res.push_back(i - m - 1);
    return res;
}
```

58 Binary Search on the Answer

AtCoder / General

Problem Statement

Pattern: "Find the minimum X such that some condition $f(X)$ holds." Directly finding X is hard, but checking $f(X)$ for a given X is easy ($O(n)$).

The Trick / Technique

If f is **monotone** (once true, always true for larger X), binary search on X. Each check takes $O(n)$; total $O(n \log(\text{range}))$.
Template: `lo = min_possible, hi = max_possible`. While `lo < hi`: `mid = (lo+hi)/2`; if `f(mid)` then `hi = mid`; else `lo = mid+1`. Answer is `lo`.

Complexity

Time: $O(n \log V)$ where V = value range **Space:** $O(1)$

```cpp
// Example: find minimum max-subarray-sum when splitting array into k groups
bool feasible(vector<int>& a, int k, long long limit) {
    int groups = 1; long long cur = 0;
    for (int x : a) {
        if (cur + x > limit) { groups++; cur = x; if (groups > k) return false; }
        else cur += x;
    }
    return true;
}
long long splitArray(vector<int>& a, int k) {
    long long lo = *max_element(a.begin(), a.end()), hi = 0;
    for (int x : a) hi += x;
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (feasible(a, k, mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}
```

59 Offline Queries with Sorting (Sweep Line) AtCoder / General

Problem Statement

Pattern: Q queries of the form "count distinct values in $[l, r]$." Online per-query is expensive; offline sorting brings it to $O((n+Q)\log n)$.

The Trick / Technique

Sort queries by right endpoint. As the right pointer sweeps from left to right, for each element $a[i]$, record its contribution only at its *latest* occurrence (use a BIT/Fenwick tree to track positions). When processing query $[l, r]$, the BIT prefix sum $[l, r]$ gives the distinct count.

Complexity

Time: $O((n+Q)\log n)$ **Space:** $O(n)$

```cpp
// BIT (Fenwick Tree)
struct BIT {
    int n; vector<int> t;
    BIT(int n) : n(n), t(n+1, 0) {}
    void update(int i, int v) { for (i++; i<=n; i+=i&-i) t[i]+=v; }
    int query(int i)          { int s=0; for (i++; i>0; i-=i&-i) s+=t[i]; return s; }
    int query(int l, int r)   { return query(r)-(l?query(l-1):0); }
};
// Count distinct in each [l,r] query
vector<int> countDistinct(vector<int>& a, vector<pair<int,int>>& qs) {
    int n=a.size(), Q=qs.size(); BIT bit(n);
    vector<int> ord(Q); iota(ord.begin(),ord.end(),0);
    sort(ord.begin(),ord.end(),[&](int a,int b){return qs[a].second<qs[b].second;});
    unordered_map<int,int> last; int ptr=0;
    vector<int> ans(Q);
    for (int qi : ord) {
        auto [l,r] = qs[qi];
        while (ptr<=r) {
            if (last.count(a[ptr])) bit.update(last[a[ptr]],-1);
            bit.update(ptr,1); last[a[ptr]]=ptr; ptr++;
        }
        ans[qi]=bit.query(l,r);
    }
    return ans;
}
```

60 Bitwise Trie for Maximum XOR Pair

AtCoder / General

Problem Statement

Find the pair (a_i, a_j) in an array that maximises $a_i \oplus a_j$.

Note to Self

This is one of those problems where a trie is the "obvious" structure only if you've seen it before. The greedy bit-by-bit construction is elegant.

The Trick / Technique

Insert all numbers into a binary trie (bit 29 to bit 0). For each number x, greedily traverse the trie choosing the opposite bit at each level (to maximise XOR). The resulting path value XOR'd with x is the maximum XOR achievable for x.

Complexity

Time: $O(n \cdot B)$ where $B = 30$ bits **Space:** $O(n \cdot B)$

```cpp
struct Trie {
    int ch[6000001][2] = {}, cnt = 1;
    void insert(int x) {
        int node = 0;
        for (int b = 29; b >= 0; b--) {
            int bit = (x >> b) & 1;
            if (!ch[node][bit]) ch[node][bit] = cnt++;
            node = ch[node][bit];
        }
    }
    int maxXOR(int x) {
        int node = 0, res = 0;
        for (int b = 29; b >= 0; b--) {
            int want = 1 - ((x >> b) & 1);  // prefer opposite bit
            if (ch[node][want]) { res |= (1 << b); node = ch[node][want]; }
            else                   node = ch[node][1 - want];
        }
        return res;
    }
};
int findMaximumXOR(vector<int>& a) {
    Trie t; for (int x : a) t.insert(x);
    int ans = 0;
    for (int x : a) ans = max(ans, x ^ t.maxXOR(x));
    return ans;
}
```

61 Two Pointer on Sorted Arrays

AtCoder / General

Problem Statement

General pattern: Given a sorted array, find a pair with sum equal to T, or count pairs with sum less than T, etc.

The Trick / Technique

Place one pointer at the start (`lo`) and one at the end (`hi`).

- If `a[lo]+a[hi] == T`: record/process, advance `lo` (or retreat `hi`).
- If `a[lo]+a[hi] < T`: advance `lo` to increase the sum.
- If `a[lo]+a[hi] > T`: retreat `hi` to decrease the sum.

Correctness: The pair space is a 2D grid; the two-pointer explores the boundary of the monotone feasibility region without missing any candidate.

Complexity

Time: $O(n)$ after sorting ($O(n \log n)$ total) **Space:** $O(1)$

```cpp
// Count pairs with sum < T (sorted array assumed)
long long countPairsLess(vector<int>& a, int T) {
    int lo = 0, hi = (int)a.size() - 1; long long cnt = 0;
    while (lo < hi) {
        if (a[lo] + a[hi] < T) { cnt += hi - lo; lo++; }  // all pairs (lo, lo+1..hi) work
        else hi--;
    }
    return cnt;
}
```

Part IV — CodeChef

10 Fundamental Mathematical Tricks

62 Fast Modular Exponentiation

CodeChef / General

Problem Statement

Compute $a^b \mod m$ efficiently for large b (up to 10^{18}).

The Trick / Technique

Binary exponentiation: $a^b = (a^{b/2})^2$ if b is even; $= a \cdot a^{b-1}$ if b is odd. Reduces b to 0 in $O(\log b)$ steps.

Complexity

Time: $O(\log b)$ **Space:** $O(1)$

```
long long power(long long a, long long b, long long m) {
    long long res = 1; a %= m;
    while (b > 0) {
        if (b & 1) res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}
```

63 Sieve of Eratosthenes

CodeChef / General

Problem Statement

Find all prime numbers up to N efficiently.

The Trick / Technique

Mark composite numbers: for each prime p, mark $p^2, p^2+p, p^2+2p, \ldots$ as composite. Start marking from p^2 (not $2p$) because smaller multiples of p were already handled by smaller primes.

Complexity

Time: $O(N \log \log N)$ **Space:** $O(N)$

```
vector<bool> sieve(int N) {
    vector<bool> is_prime(N+1, true);
    is_prime[0] = is_prime[1] = false;
    for (int p = 2; (long long)p*p <= N; p++)
        if (is_prime[p])
            for (int j = p*p; j <= N; j += p) is_prime[j] = false;
    return is_prime;
}
```

64 Smallest Prime Factor Sieve CodeChef / General

Problem Statement

Preprocess so that any number $n \leq N$ can be **fully factorized** in $O(\log n)$ (after $O(N \log \log N)$ preprocessing).

The Trick / Technique

Run a modified sieve that stores the **smallest prime factor** (SPF) of every number. To factorize n: divide by `spf`$[n]$ repeatedly. This is far faster than trial division for many queries.

Complexity

Build: $O(N \log \log N)$ **Factorize:** $O(\log n)$ per query

```cpp
vector<int> buildSPF(int N) {
    vector<int> spf(N+1); iota(spf.begin(), spf.end(), 0);
    for (int i = 2; (long long)i*i <= N; i++)
        if (spf[i] == i)  // i is prime
            for (int j = i*i; j <= N; j += i)
                if (spf[j] == j) spf[j] = i;
    return spf;
}
map<int,int> factorize(int n, vector<int>& spf) {
    map<int,int> f;
    while (n > 1) { f[spf[n]]++; n /= spf[n]; }
    return f;
}
```

65 Modular Inverse

CodeChef / General

Problem Statement

Compute $a^{-1} \mod m$ (the modular multiplicative inverse) so that $a \cdot a^{-1} \equiv 1 \pmod{m}$.

The Trick / Technique

Method 1 (Fermat's Little Theorem): If m is prime, $a^{-1} = a^{m-2} \bmod m$. Use fast exponentiation.
Method 2 (Extended GCD): Works for any coprime a, m. Solve $ax + my = 1$; the x value is the inverse. x may be negative; add m if needed.

Complexity

Fermat: $O(\log m)$ (prime m only). **ExtGCD:** $O(\log m)$ (any m).

```
// Fermat (prime modulus only)
long long modInv(long long a, long long m) { return power(a, m - 2, m); }

// Extended GCD (works for any coprime a, m)
long long extGCD(long long a, long long b, long long& x, long long& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    long long x1, y1, g = extGCD(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1;
    return g;
}
long long modInvGeneral(long long a, long long m) {
    long long x, y; extGCD(a, m, x, y);
    return (x % m + m) % m;
}
```

66 Number of Divisors

CodeChef / General

Problem Statement

Count the number of divisors of n given its prime factorization.

The Trick / Technique

If $n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$, then the number of divisors is $(e_1 + 1)(e_2 + 1) \cdots (e_k + 1)$.
Why? Each divisor independently chooses an exponent in $[0, e_i]$ for each prime p_i. The choices are independent, so the count multiplies.

Complexity

Time: $O(\sqrt{n})$ for trial-division factorization, or $O(\log n)$ with SPF sieve.

```cpp
long long countDivisors(long long n) {
    long long cnt = 1;
    for (long long p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            int exp = 0;
            while (n % p == 0) { exp++; n /= p; }
            cnt *= (exp + 1);
        }
    }
    if (n > 1) cnt *= 2;  // n is a prime factor with exponent 1
    return cnt;
}
```

67 Brian Kernighan Bit Tricks

CodeChef / General

Problem Statement

Efficiently: (1) Count set bits in n. (2) Check if n is a power of two. (3) Isolate the lowest set bit.

The Trick / Technique

$n \,\&\, (n-1)$ **clears the lowest set bit** of n.

- **Count set bits:** Repeatedly apply until $n = 0$; count iterations. O(number of set bits) — faster than checking all bits.
- **Power of two:** $n > 0$ and $n \,\&\, (n-1) == 0$.
- **Lowest set bit:** $n \,\&\, (-n)$ (two's complement).

Complexity

Time: O(popcount) for bit count, $O(1)$ for others.

```
int countBits(int n) { int c = 0; while (n) { n &= n-1; c++; } return c; }
bool isPow2(int n)   { return n > 0 && (n & (n-1)) == 0; }
int  lowestBit(int n){ return n & (-n); }
int  highestBit(int n){ return 1 << (31 - __builtin_clz(n)); }
```

68 Modular Arithmetic Pitfalls

CodeChef / General

Problem Statement

You are computing $(A \cdot B + C - D) \bmod M$ where values can be very large or negative. How do you handle it correctly?

The Trick / Technique

Key identities (all correct when $a, b \geq 0$): $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
$(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$
Pitfall — Negative results: In C++, $(-1)\%m$ is -1 (not $m - 1$). Always write `((x % m) + m) % m` when subtraction is involved.
Pitfall — Overflow: $(a \bmod m) \cdot (b \bmod m)$ can overflow `int` if $m \approx 10^9$. Use `long long` for intermediate products.

Complexity

Remember: Division requires modular inverse; you cannot simply do $(a/b) \bmod m$.

```
const long long MOD = 1e9 + 7;
long long safeAdd(long long a, long long b) { return (a + b) % MOD; }
long long safeSub(long long a, long long b) { return ((a - b) % MOD + MOD) % MOD; }
long long safeMul(long long a, long long b) { return a % MOD * (b % MOD) % MOD; }
long long safeDiv(long long a, long long b) { return safeMul(a, power(b, MOD-2, MOD)); }
```

69 GCD Properties and Applications

CodeChef / General

Problem Statement

Know: (1) Euclidean GCD. (2) LCM. (3) GCD of an array. (4) Number of integers in $[1, n]$ coprime to m (Euler's totient). (5) GCD of all subarray XORs.

The Trick / Technique

Euclidean: $\gcd(a, b) = \gcd(b, a \bmod b)$. Base: $\gcd(a, 0) = a$.
LCM: $\text{lcm}(a, b) = a / \gcd(a, b) \cdot b$ (divide first to avoid overflow).
Array GCD: $\gcd(a_1, a_2, \ldots, a_n)$ — just fold left.
GCD array trick: $\gcd(a_l, a_{l+1}, \ldots, a_r) = \gcd(\text{prefix}[r], \text{prefix}[l-1])$ where $\text{prefix}[i] = \gcd(a_0, \ldots, a_i)$? **NO** — prefix GCD is non-increasing and GCD is not easily range-queryable with prefix arrays. Use sparse table on GCD instead.

Complexity

GCD: $O(\log \min(a, b))$ **Array GCD:** $O(n \log V)$ **Sparse Table GCD:** $O(n \log n)$ build, $O(1)$ query

```
long long gcd(long long a, long long b) { return b ? gcd(b, a%b) : a; }
long long lcm(long long a, long long b) { return a / gcd(a,b) * b; }
long long arrayGCD(vector<int>& a) {
    long long g = a[0]; for (int x : a) g = gcd(g, x); return g;
}
```

70 Inclusion-Exclusion Principle CodeChef / General

Problem Statement

Count integers in $[1, N]$ divisible by at least one of $p_1, p_2, \ldots, p_k$.

The Trick / Technique

$|A_1 \cup A_2 \cup \cdots \cup A_k| = \sum |A_i| - \sum |A_i \cap A_j| + \cdots \pm |A_1 \cap \cdots \cap A_k|$
For divisibility: $|A_i \cap A_j| = \lfloor N / \mathrm{lcm}(p_i, p_j) \rfloor$. Enumerate all 2^k non-empty subsets; add or subtract based on parity of subset size.

Complexity

Time: $O(2^k \cdot k)$ for small k **Space:** $O(1)$

```cpp
// Count integers in [1,N] divisible by at least one of primes[]
long long inclusionExclusion(long long N, vector<long long>& primes) {
    int k = primes.size();
    long long ans = 0;
    for (int mask = 1; mask < (1 << k); mask++) {
        long long lcmVal = 1; int bits = __builtin_popcount(mask);
        for (int i = 0; i < k; i++)
            if (mask >> i & 1) { lcmVal = lcm(lcmVal, primes[i]); if (lcmVal > N) break; }
        if (bits & 1) ans += N / lcmVal;
        else          ans -= N / lcmVal;
    }
    return ans;
}
```

71 Pigeonhole Principle — Recognising When It Applies

CodeChef / General

Problem Statement

Examples of PHP in problems:
(1) Array of $n+1$ integers in $[1, n] \Rightarrow$ at least one duplicate exists.
(2) Any $n+1$ integers contain two with the same remainder mod n.
(3) In a graph of n vertices, if every vertex has degree $\geq n/2$, a Hamiltonian path exists.
What is the general pattern?

The Trick / Technique

Pigeonhole Principle: If $n+1$ objects are placed into n boxes, at least one box contains ≥ 2 objects.
How to spot it:

- The problem guarantees existence without telling you where.
- There's a mismatch between the number of "objects" and "categories."
- The word "at least one" or "must exist" appears in the conclusion.

Using it algorithmically: Instead of searching, use PHP to *prove* your answer is correct or to bound the search space. For finding the duplicate: cycle detection (LC 287) is PHP applied constructively.

Complexity

PHP is a **proof technique**, not an algorithm. The algorithm that leverages it (cycle detection, birthday paradox, random sampling) has its own complexity.

```cpp
// PHP in practice: random sampling for element with frequency > n/3
// By PHP, at most 2 such elements exist. Random sample ~9 elements;
// probability of missing both is (2/3)^9 < 3% per trial.
int findMajorityRandom(vector<int>& a, int threshold) {
    mt19937 rng(42);
    int n = a.size();
    for (int trial = 0; trial < 20; trial++) {
        int cand = a[rng() % n];
        int cnt  = count(a.begin(), a.end(), cand);
        if (cnt * threshold > n) return cand;
    }
    return -1;  // no majority element
}
```